FORMATO DE DESARROLLO E INTEGRACIONES

FT-SL-004-Módulo_Cartera

STF GROUP S.A.
INFORMATICA – Py. CREDI10 SOFTWARE PROPIO

Cali, lunes 12 de febrero del 2024

TABLA DE CONTENIDO

INTRODUCCION 3
1. DETALLE TECNICO 4
1.1 Detallar componentes existentes..... 5
1.2 Detallar funcionalidades 5
1.3 Observaciones y/o Comentarios..... 37
2. CASOS DE PRUEBAS UNITARIAS 38

INTRODUCCION

Este documento muestra en detalle todas las actividades de la solución a desarrollar con sus dependencias, entradas y salidas. Así como los casos de pruebas unitarias realizadas a la solución.

NOTA: El desarrollo de cualquier solución de requerimiento debe seguir los estándares de desarrollo que define Arquitectura.

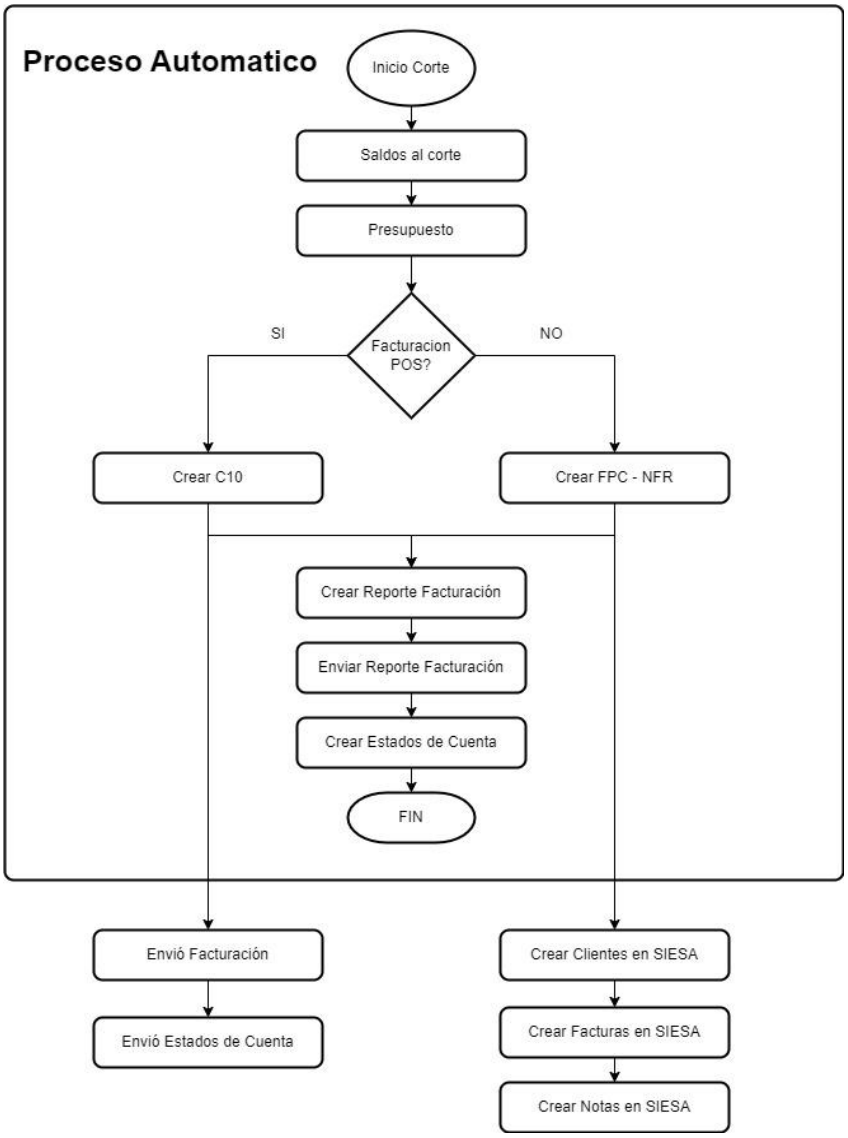
1. DETALLE TECNICO

1.1 Descripción del Requerimiento:

1.1.1 Requerimiento

Realizar los procesos saldos al corte, presupuesto, facturación, estados de cuenta y creación del reporte de facturas en las fechas adecuadas para los cortes establecidos por credi10.

1.1.2 Flujo del Proceso

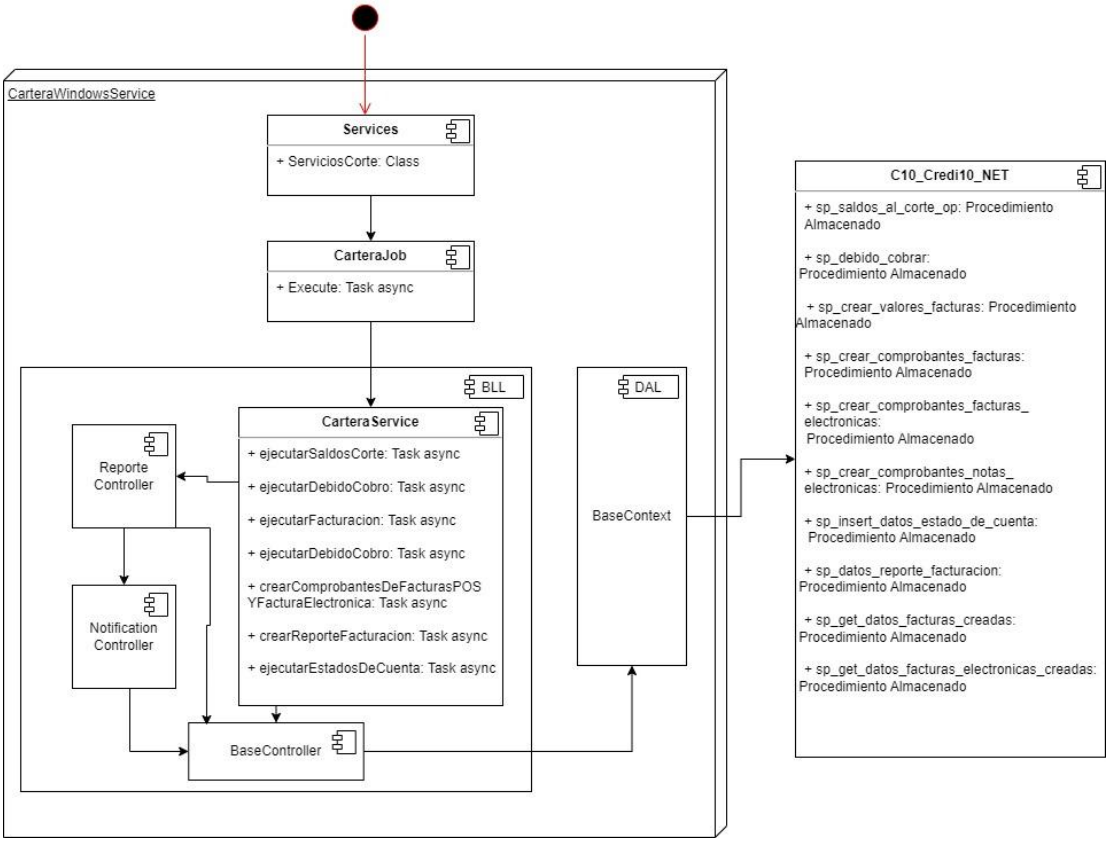


1.2 Detallar componentes existentes

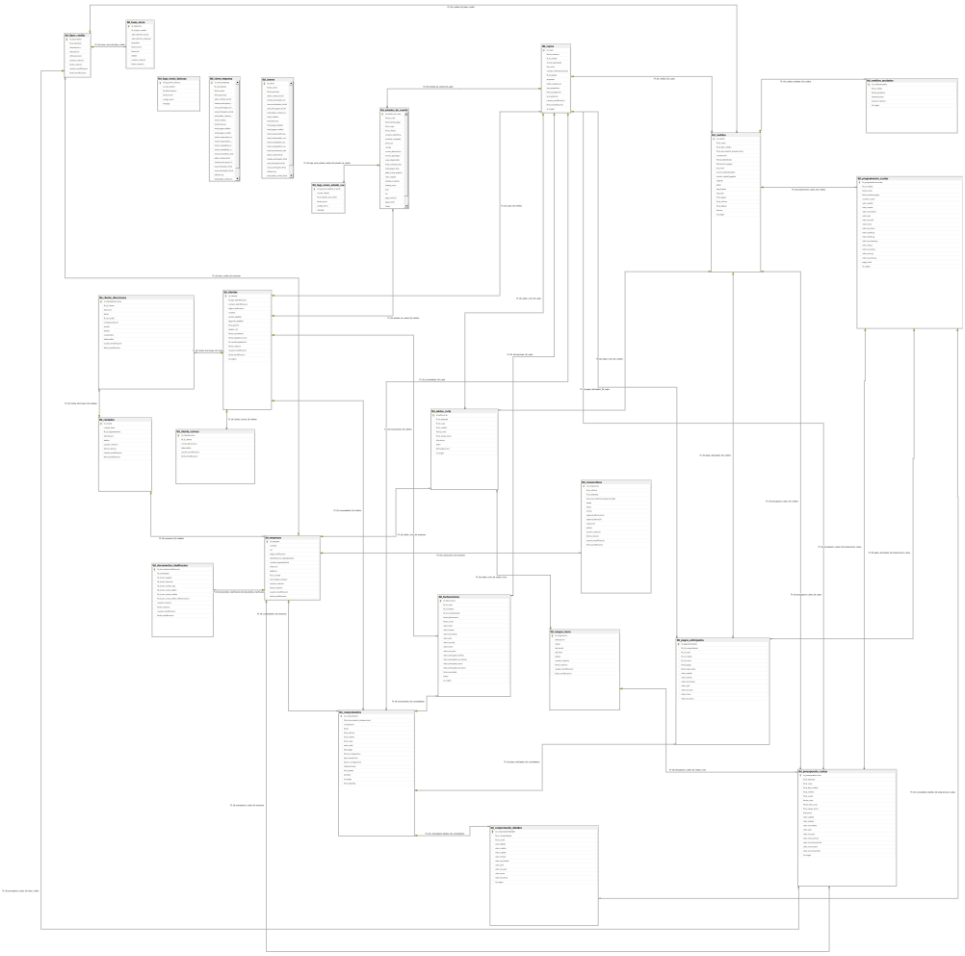
SOLUCION	
CarteraWindowsServices	Proyecto de tipo servicio de Windows que se ejecuta automáticamente en fechas establecidas que realiza el proceso de cada corte en credi10.
Creditos	Proyecto Web que ejecuta procesos después del corte automático.
Quartz.NET	Librería que ejecuta tareas en segundo plano en una hora y fecha determinada.
BASE DE DATOS	
C10_Credi10_NET	Tablas y Procedimientos almacenados

1.3 Detallar funcionalidades

1.3.1 Diagrama de Componentes



1.3.2 MER



Para una mejor visualización, consultar el MER aqui.

1.3.3 Parametrizaciones

Descripción	Grupo	Valor
Correo origen estado cuenta	Email	credi10estadocuenta@credi10.com.co
Valor IdEmpresa para crear la facturación	Cartera	1
Correo al que le enviara el reporte de la facturación creada	Facturacion	Correo
Ruta archivos de reportes de facturación	Facturacion	\\COATLANTA\Credi10\ReporteFactura\

Url Estados de cuenta clientes	EstadosDeCuenta	https://www.credi10.com.co/EstadosCuenta/EstadoCuentaPersonalizado/
Url Facturas clientes	Facturacion	https://www.credi10.com.co/Facturas/FacturaPersonalizada/
Parámetro WS SIESA - WSDL	FacturacionElectronica	http://192.168.0.xxx/WSUNOEE/WSUNOEE.asmx
Parámetro WS SIESA - Usuario	FacturacionElectronica	interfaz_ecommerce
Parámetro WS SIESA - Clave	FacturacionElectronica	LvH4GYi3
Parámetro WS SIESA - Nombre de Conexión	FacturacionElectronica	STFColombia
Notas de Reclasificación de Saldos	FacturacionElectronica	NCR
Cantidad UVT Mínimo Facturacion Elec.	FacturacionElectronica	5
Valor UVT Vigente	FacturacionElectronica	47065 (Se debe actualizar cada año)
Encendido o apagado de Facturacion Electrónica	FacturacionElectronica	true
Ítem en el ERP para Intereses Corrientes	FacturacionElectronica	0250890
Ítem en el ERP para Intereses Mora	FacturacionElectronica	0251372
Ítem en el ERP para AVAL	FacturacionElectronica	0251373
Es posible enviar estados de cuenta	Estados de Cuenta	Es posible enviar Estado de cuenta para el corte
Ya se envió estados de cuenta anteriormente	Estados de Cuenta	El proceso de envió del estado de cuenta ya se realizó anteriormente
No se puede enviar estados de cuenta para la fecha ingresada	Estados de Cuenta	La fecha indicada no es correcta para el proceso de envió de estados de cuenta
No se puede enviar estados de cuenta	Estados de Cuenta	No es posible enviar estados de cuenta

porque no se ha enviado la facturacion		porque no se ha enviado la facturación del corte
HTML para envió de la factura POS al cliente	Email	Html....
HTML para reporte de facturacion creada	Email	Html....
HTML para envió estado de cuenta link	Email	Html....

1.3.4 Proceso

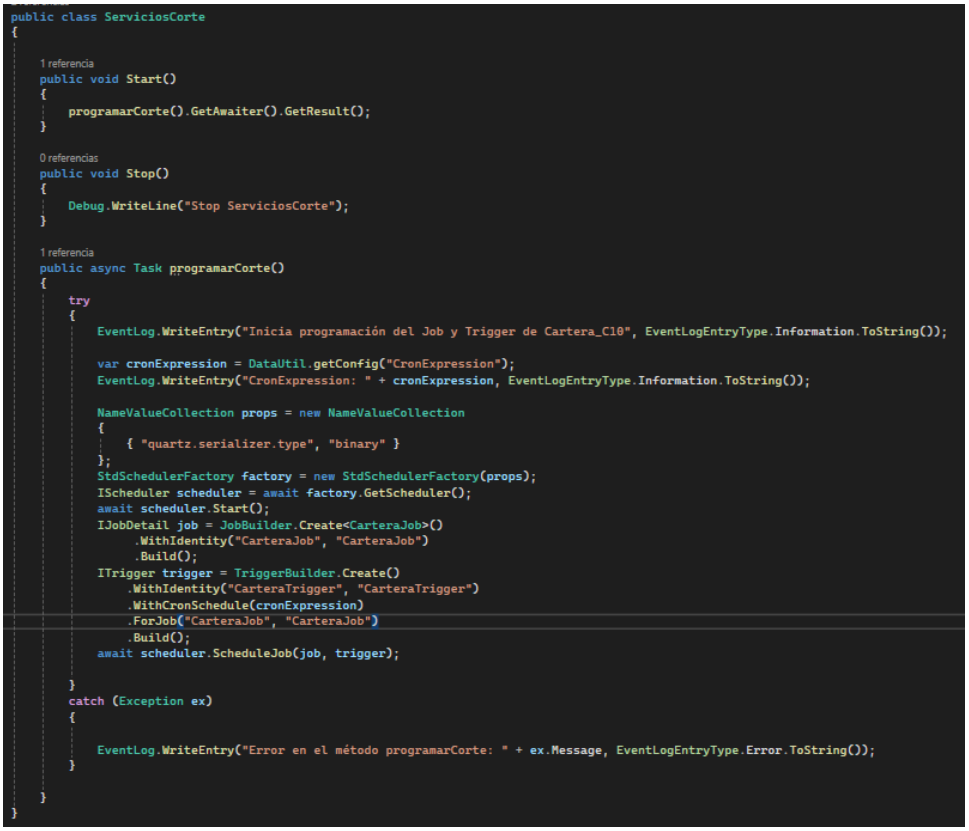
- **Proceso automático en cada corte**

Para cumplir con el requerimiento de Cartera se creó una solución que incluye un servicio de Windows que ejecutara el proceso requerido para cada corte como una tarea en segundo plano.

Para esto en el proyecto se utilizó la librería Quartz.NET que se encarga de ejecutar tareas en segundo plano en un momento determinado de manera precisa y confiable. Quartz.NET maneja 3 conceptos utilizados en la solución.

- El Job: Son tareas en segundo plano que se pueden ejecutar.
- El Trigger: Controla cuándo se ejecuta un Job y se activa en algún tipo de programa.
- El Scheduler: Coordina los Jobs y Triggers, ejecutando los Jobs según lo requieran los Triggers.

Estos 3 conceptos son configurados en el método programarCorte y como se explicó anteriormente con cada concepto, este será el momento inicial donde se establece que tareas o procesos se ejecutaran y en qué día y hora especifico lo hará. El método en mención programara cuando se instale e inicie el servicio que la tarea que se tendrá que ejecutar se llamara CarteraJob y el momento en que se debería ejecutar estará establecido por un CronExpression que se indique en su configuración.



```
public class ServiciosCorte
{

    1 referencia
    public void Start()
    {
        programarCorte().GetAwaiter().GetResult();
    }

    0 referencias
    public void Stop()
    {
        Debug.WriteLine("Stop ServiciosCorte");
    }

    1 referencia
    public async Task programarCorte()
    {
        try
        {
            EventLog.WriteEntry("Inicia programación del Job y Trigger de Cartera_C10", EventLogEntryType.Information.ToString());

            var cronExpression = DataUtil.getConfig("CronExpression");
            EventLog.WriteEntry("CronExpression: " + cronExpression, EventLogEntryType.Information.ToString());

            NameValueCollection props = new NameValueCollection
            {
                { "quartz.serializer.type", "binary" }
            };
            StdSchedulerFactory factory = new StdSchedulerFactory(props);
            IScheduler scheduler = await factory.GetScheduler();
            await scheduler.Start();
            IJobDetail job = JobBuilder.Create<CarteraJob>()
                 .WithIdentity("CarteraJob", "CarteraJob")
                 .Build();
            ITrigger trigger = TriggerBuilder.Create()
                .WithIdentity("CarteraTrigger", "CarteraTrigger")
                .WithCronSchedule(cronExpression)
                .ForJob("CarteraJob", "CarteraJob")
                .Build();
            await scheduler.ScheduleJob(job, trigger);

        }
        catch (Exception ex)
        {

            EventLog.WriteEntry("Error en el método programarCorte: " + ex.Message, EventLogEntryType.Error.ToString());
        }

    }
}
```

El Cron Expression especifica un patrón de tiempo para programar tareas en sistemas y en la solución es utilizado en la configuración inicial del servicio. Esta parametrizado en un archivo XML llamado *ConfiguracionCartera.xml* ubicado en el proyecto. El valor por defecto es (**0 0 1 1,16 * ? ***) y se desglosa de la siguiente manera

- **Segundos (0):** La tarea se ejecutará cuando el reloj marque cero segundos.
- **Minutos (0):** La tarea se ejecutará cuando el reloj marque cero minutos.
- **Horas (1):** La tarea se ejecutará a la hora 1 (1 AM).
- **Día del mes (1,16):** La tarea se ejecutará el día 1 y 16. Se definieron estos días ya que en credi10 existen dos cortes, el 15 y 30 (último día del mes en realidad).
- **Mes (*):** Este campo se establece como un comodín (*), lo que significa que la tarea se ejecutará independientemente del mes.
- **Día de la semana (?):** Este campo se establece como un comodín (?), lo que significa que no se especifica un día de la semana en particular.
- **Año (*):** Este campo se establece como un comodín (*), lo que significa que la tarea se ejecutará independientemente del año.

```xml
<?xml version="1.0" encoding="utf-8" ?>
<configuracion>
    <add key="CronExpression" value="0 0 1 1,16 * ? *" />
</configuracion>
```

Como se menciona anteriormente el Job creado para que se ejecute se llama CarteraJob y este tendrá un método llamado Execute que dentro de él se tendrá los diferentes métodos que se deberán ejecutar para realizar el proceso de corte en credi10.

```
public class CarteraJob : IJob
{
    0 referencias
    public async Task Execute(IJobExecutionContext context)
    {
        var cartera = new CarteraService();
        EventLog.WriteEntry("Inicio del proceso saldos al corte C10", EventLogEntryType.Information.ToString());
        await cartera.ejecutarSaldosCorte();
        EventLog.WriteEntry("Fin del proceso saldos al corte C10", EventLogEntryType.Information.ToString());

        EventLog.WriteEntry("Inicio del proceso debido cobro C10", EventLogEntryType.Information.ToString());
        await cartera.ejecutarDebidoCobro();
        EventLog.WriteEntry("Fin del proceso debido cobro C10", EventLogEntryType.Information.ToString());

        EventLog.WriteEntry("Inicio del proceso crear valores facturas pos y facturas electronicas", EventLogEntryType.Information.ToString());
        await cartera.ejecutarFacturacion();
        EventLog.WriteEntry("Fin del proceso crear valores facturas pos y facturas electronicas", EventLogEntryType.Information.ToString());

        EventLog.WriteEntry("Inicio del proceso crear comprobantes de facturas C10 y facturas FPC", EventLogEntryType.Information.ToString());
        await cartera.crearComprobantesDeFacturasPOSYFacturaElectronica();
        EventLog.WriteEntry("Fin del proceso crear valores facturas C10 y facturas FPC", EventLogEntryType.Information.ToString());

        EventLog.WriteEntry("Inicio del proceso crear reporte", EventLogEntryType.Information.ToString());
        await cartera.crearReporteFacturacion();
        EventLog.WriteEntry("Fin del proceso crear y enviar reporte", EventLogEntryType.Information.ToString());

        EventLog.WriteEntry("Inicio del proceso creacion de estados de cuenta", EventLogEntryType.Information.ToString());
        await cartera.ejecutarEstadosDeCuenta();
        EventLog.WriteEntry("Fin del proceso creacion de estados de cuenta", EventLogEntryType.Information.ToString());
    }
}
```

Todos los métodos que son llamados en el Job son tareas asíncronas, pero al utilizar el parámetro await en el llamado de cada una se convierten dentro del Job en tareas síncronas, esto se hace así ya que cada proceso depende de uno anterior, por lo que se debe garantizar que se ejecute un proceso a la vez y apenas finalice uno empiece el otro.

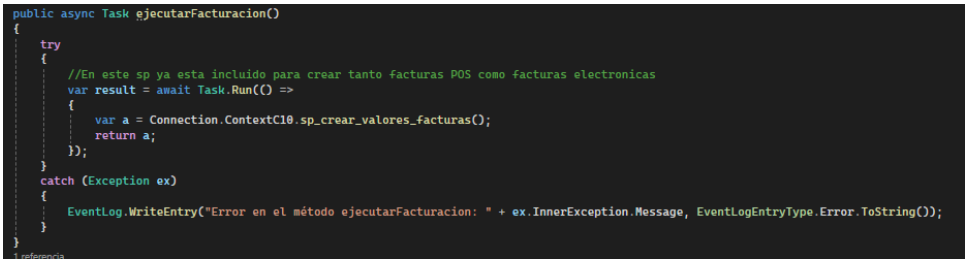
```
1 referencia
public async Task ejecutarSaldosCorte()
{
    try
    {
        var result = await Task.Run(() =>
        {
            var a = Connection.ContextC10.sp_saldos_al_corte_op();
            return a;
        });
    }
    catch (Exception ex)
    {
        EventLog.WriteEntry("Error en el método ejecutarSaldosCorte: " + ex.InnerException.Message, EventLogEntryType.Error.ToString());
    }
}
```

El primer proceso que se ejecuta es la tarea llamada *ejecutarSaldosCorte()*, este método llamara internamente al procedimiento almacenado *sp_saldos_al_corte_op()* que es el encargado de manejar toda la lógica para la creación de los saldos en cada corte.



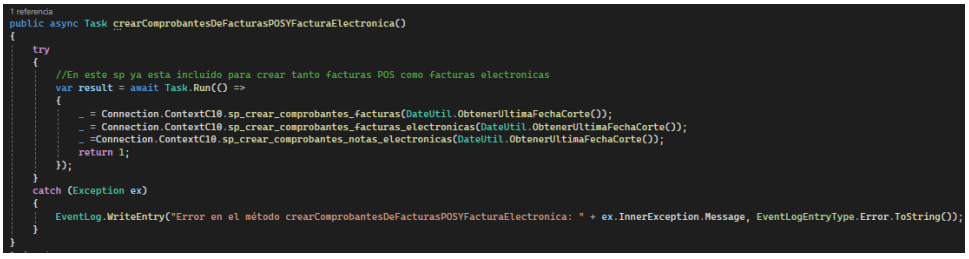
```
1 referencia
public async Task ejecutarDebidoCobro()
{
    try
    {
        var result = await Task.Run(() =>
        {
            var a = Connection.ContextC10.sp_debido_cobrar();
            return a;
        });
    }
    catch (Exception ex)
    {
        EventLog.WriteEntry("Error en el método ejecutarDebidoCobro: " + ex.InnerException.Message, EventLogEntryType.Error.ToString());
    }
}
```

El segundo proceso que se ejecuta es la tarea llamada *ejecutarDebidoCobro()* este método llamara internamente al procedimiento almacenado *sp_debido_cobrar()* que es el encargado de manejar toda la lógica para la creación de los presupuestos de todas las cuotas que a la fecha se les debe realizar el corte respectivo, los valores que se presupuesten aquí será los valores que se le deberá cobrar al cliente.



```
public async Task ejecutarFacturacion()
{
    try
    {
        //En este sp ya esta incluido para crear tanto facturas POS como facturas electronicas
        var result = await Task.Run(() =>
        {
            var a = Connection.ContextC10.sp_crear_valores_facturas();
            return a;
        });
    }
    catch (Exception ex)
    {
        EventLog.WriteEntry("Error en el método ejecutarFacturacion: " + ex.InnerException.Message, EventLogEntryType.Error.ToString());
    }
}
1 referencia
```

El tercer proceso que se ejecuta es la tarea llamada *ejecutarFacturacion()*, este método llamara internamente al procedimiento almacenado *sp_crear_valores_facturas()* que es el encargado de manejar toda la lógica para la creación de lo que es facturacion POS y/o facturacion electrónica, el sp dependiendo del caso registrara en las tbl_facturaciones y/o tbl_facturacion_electronica los valores separados por los diferentes conceptos.



```
1 referencia
public async Task crearComprobantesDeFacturasPOSYFacturaElectronica()
{
    try
    {
        //En este sp ya esta incluido para crear tanto facturas POS como facturas electronicas
        var result = await Task.Run(() =>
        {
            _ = Connection.ContextC10.sp_crear_comprobantes_facturas(DateUtil.ObtenerUltimaFechaCorte());
            _ = Connection.ContextC10.sp_crear_comprobantes_facturas_electronicas(DateUtil.ObtenerUltimaFechaCorte());
            _ =Connection.ContextC10.sp_crear_comprobantes_notas_electronicas(DateUtil.ObtenerUltimaFechaCorte());
            return 1;
        });
    }
    catch (Exception ex)
    {
        EventLog.WriteEntry("Error en el método crearComprobantesDeFacturasPOSYFacturaElectronica: " + ex.InnerException.Message, EventLogEntryType.Error.ToString());
    }
}
```

El cuarto proceso que se ejecuta es la tarea llamada *crearComprobantesDeFacturasPOSYFacturaElectronica()*, este método llamara internamente tres procedimientos almacenados *sp_crear_comprobantes_facturas()* que es el encargado de crear los documentos C10 por si se crearon facturas POS en el corte y llamara al *sp_crear_comprobantes_facturas_electronicas()* junto a *sp_crear_comprobantes_notas_electronicas()* que serán los encargados de crear

los documentos FPC y NFR necesarias para las facturas electronicas que deben ser enviadas a Siesa.

```
1 referencia
public async Task crearReporteFacturacion()
{
    try
    {
        await Reporte.CrearReporteExcelFacturacionAsync();
    }
    catch (Exception ex)
    {
        EventLog.WriteEntry("Error en el método crearReporteFacturacion: " + ex.InnerException.Message, EventLogEntryType.Error.ToString());
    }
}
```

El quinto proceso que se ejecuta es la tarea llamada *crearReporteFacturacion()*, este método llamara internamente al método encontrado en la clase Reporte llamado *CrearReporteExcelFacturacionAsync()* que es el encargado de manejar toda la lógica para la creación del reporte y posterior envío de los valores que se facturaron para el corte que se está realizando.

```
try
{
    var listaFacturas = Connection.ContextC10.sp_get_datos_facturas_creadas().ToList();
    var datos = Connection.ContextC10.sp_datos_reporte_facturacion().First();

    var datosReporte = new DatosReporte
    {
        RutaArchivo = datos.RutaArchivo,
        Asunto = datos.Asunto,
        CorreoDe = datos.CorreoDe,
        CorreoPara = datos.CorreoPara,
        EmailHost = datos.EmailHost,
        EmailKeyToken = datos.EmailKeyToken,
        EmailValueToken = datos.EmailValueToken,
        FechaCorte = (DateTime)datos.FechaCorte,
        MensajeHtml = datos.MensajeHtml
    };
```

```
//FacturaElectronica
oSLDocument.AddWorksheet("FacturaElectronica");

var listaFacturasElectronicas = Connection.ContextC10.sp_get_datos_facturas_electronicas_creadas().ToList();
```

En la creación del reporte se utilizan 3 procedimientos almacenados, el primero llamado sp_datos_reporte_facturacion que traerá la información necesaria para que el sistema conozca la ruta donde debe ser guardado y la información para poder enviar el reporte por correo.

Los dos otros procedimientos almacenados son dos consultas que traerán el listado de todas las facturas POS y electronicas creadas para el corte con el detalle de cada uno.



```
1 referencia
public async Task ejecutarEstadosDeCuenta()
{
    try
    {
        var result = await Task.Run(() =>
        {
            var fechaActual = DateTime.Today;
            var a = Connection.ContextC10.sp_insert_datos_estado_de_cuenta(fechaActual);
            return a;
        });
    }
    catch (Exception ex)
    {
        EventLog.WriteEntry("Error en el método ejecutarEstadosDeCuenta: " + ex.InnerException.Message, EventLogEntryType.Error.ToString());
    }
}
```

El último proceso que se ejecuta es la tarea llamada *ejecutarEstadosDeCuenta()*, este método llamara internamente al procedimiento almacenado *sp_insert_datos_estado_de_cuenta()* que es el encargado de manejar toda la lógica para la creación de los estados de cuenta.

Estos son todos los procesos que se deben ejecutar sin excepción para cada corte de manera automática, los procedimientos almacenados que cargan con toda la lógica de los procesos son explicados a detalle en este punto del documento. Lo siguiente es el proceso de envió de las facturaciones y estados de cuenta que serán documentados y explicados en el documento ProcesoEnvioFacturacionYEstadosDeCuenta.docx y el cierre de cartera que será explicado en el documento CierreCartera.docx.

1.3.5 Documentación de Arquitectura

Flujogramas Cartera

1.3.6 Log

Los logs que se registran en la solución *CarteraWindowsService* y en la solución *Creditos* para seguir una auditoria son los siguientes:

Acción	Detalle	Solución
Inicia programación del Job y Trigger de Cartera_C10	Registra en los eventos de Windows el inicio del servicio creado.	CarteraWindowsService
CronExpression: ########	Registra en los eventos de Windows el valor del Cron Expression que indica cada cuanto se ejecutara el servicio.	CarteraWindowsService
Error en el método programarCorte:	Registra en los eventos de Windows si hubo un error en la configuración	CarteraWindowsService

	del método que ejecutara el servicio llamado programarCorte de ServiciosCorte.cs.	
Stop ServiciosCorte	Registra en los eventos de Windows cuando se detiene la ejecución del servicio.	CarteraWindowsService
Inicio del proceso saldos al corte C10	Registra en los eventos de Windows cuando inicia el proceso de creación de los saldos al corte.	CarteraWindowsService
Fin del proceso saldos al corte C10	Registra en los eventos de Windows cuando finaliza el proceso de creación de los saldos al corte.	CarteraWindowsService
Inicio del proceso debido cobro C10	Registra en los eventos de Windows cuando inicia el proceso de creación del presupuesto al corte.	CarteraWindowsService
Fin del proceso debido cobro C10	Registra en los eventos de Windows cuando finaliza el proceso de creación del presupuesto al corte.	CarteraWindowsService
Inicio del proceso crear valores facturas pos y facturas electronicas	Registra en los eventos de Windows cuando inicia el proceso de crear los valores de las facturas en el corte.	CarteraWindowsService
Fin del proceso crear valores facturas pos y facturas electronicas	Registra en los eventos de Windows cuando finaliza el proceso de crear los valores de las facturas en el corte.	CarteraWindowsService
Inicio del proceso crear comprobantes de facturas C10 y facturas FPC	Registra en los eventos de Windows cuando inicia el proceso de creación de los comprobantes	CarteraWindowsService

	respectivos a las facturas (C10 o FPC y NCR).	
Fin del proceso crear valores facturas C10 y facturas FPC	Registra en los eventos de Windows cuando finaliza el proceso de creación de los comprobantes respectivos a las facturas (C10 o FPC y NCR).	CarteraWindowsService
Inicio del proceso crear reporte	Registra en los eventos de Windows cuando inicia el proceso de creación del reporte y él envió respectivo.	CarteraWindowsService
Fin del proceso crear y enviar reporte	Registra en los eventos de Windows cuando finaliza el proceso de creación del reporte y él envió respectivo.	CarteraWindowsService
Inicio del proceso creación de estados de cuenta	Registra en los eventos de Windows cuando inicia el proceso que crea los estados de cuenta.	CarteraWindowsService
Fin del proceso creación de estados de cuenta	Registra en los eventos de Windows cuando finaliza el proceso que crea los estados de cuenta.	CarteraWindowsService
Error en el método ejecutarSaldosCorte:	Registra en los eventos de Windows si hubo un error en el método que realiza la ejecución del procedimiento almacenado.	CarteraWindowsService
Error en el método ejecutarDebidoCobro:	Registra en los eventos de Windows si hubo un error en el método que realiza la ejecución del procedimiento almacenado.	CarteraWindowsService
Error en el método ejecutarFacturacion:	Registra en los eventos de Windows si hubo un error en el método que	CarteraWindowsService

	realiza la ejecución del procedimiento almacenado.	
Error en el método crearComprobantesDeFacturasPOSYFacturaElectronica:	Registra en los eventos de Windows si hubo un error en el método que realiza la ejecución del procedimiento almacenado.	CarteraWindowsService
Error en el método crearReporteFacturacion:	Registra en los eventos de Windows si hubo un error en el método que realiza la creación del archivo.	CarteraWindowsService
Error en el método ejecutarEstadosDeCuenta:	Registra en los eventos de Windows si hubo un error en el método que realiza la ejecución del procedimiento almacenado.	CarteraWindowsService
Reporte Creado Correctamente	Registra en los eventos de Windows si la creación del reporte de factura se realizó correctamente.	CarteraWindowsService
Reporte Enviado	Registra en los eventos de Windows si él envió del reporte de factura se realizó correctamente.	CarteraWindowsService
Error creando el reporte:	Registra en los eventos de Windows si hubo un error al crear el reporte de facturas.	CarteraWindowsService
Error enviando el archivo	Registra en los eventos de Windows si hubo un error al enviar el reporte de facturas.	CarteraWindowsService

Todos los logs del proyecto CarteraWindowsService se registran en los eventos de Windows y estos podrán verse abriendo el visor de eventos encontrado en el servidor donde estará desplegado la aplicación.

1.3.7 Lista de modificaciones hacia la base de datos

Script 1. Creación de tablas: Para crear el esquema completo de la base de datos de la aplicación.

Creacion_BD_C10_Credi10Net.sql

1.3.8 Lista de Script´s y la descripción de lo que contiene cada uno.

- Ejecutar el script sp_saldos_al_corte_op.sql que contiene la creación del procedimiento almacenado que permite establecer el saldo pendiente de cada crédito, determinando si tiene mora y clasificándola en rangos.

```
Declare @ValidacionCorte Table(PermitirProceso bit,FechaCorte date, FechaEjecucion date)
    Insert @ValidacionCorte Exec sp_validar_corte_inicio_mes

select  @permitir = PermitirProceso from @ValidacionCorte
if @permitir = 1
begin...
end
```

El proceso de corte solo es permitido realizarlo en unas fechas determinadas por la lógica de negocio, para esto se implementó un procedimiento almacenado llamado sp_validar_corte_inicio_mes.sql que validara si la fecha en que se está ejecutando el proceso es la correcta, pues este devolverá el permiso y la fecha de corte válida para ejecutar el proceso.

```
SELECT cr.id_credito, emp.id_empresa, cr.fk_id_cupo,
       (cr.monto_desembolsado -cr.monto_capital_pagado) as saldo,
       id_programacioncuotas, fk_id_credito, fecha_corte, fecha_maxima_pago, numero_cuota,
       valor_capital, valor_interes, valor_iva_interes, valor_aval, valor_iva_aval,
       valor_mora, valor_iva_mora, valor_capital_p, valor_interes_p, valor_iva_interes_p,
       valor_aval_p, valor_iva_aval_p, valor_mora_p, valor_iva_mora_p, pago_total
        into #tempJoin FROM tbl_creditos as cr
            join tbl_oficinas as ofi on CR.fk_id_oficina = ofi.id_oficina
            join tbl_marcas as marca on marca.id_marca = ofi.fk_id_marca
            join tbl_empresas as emp on emp.id_empresa = marca.fk_id_empresa
            join tbl_programacion_cuotas as cu on  cu.fk_id_credito = cr.id_credito
              where cr.vigente = 1
                    and cr.dia_corte =  @diaCorte15o30
                    and cu.fecha_corte <= @fechaCorte
                    and cr.fk_id_estado = 1
                    and(cu.pago_total is null or cu.pago_total = 0)
                    and numero_cuota <> 0
                    or (cr.monto_capital_pagado < cr.monto_desembolsado and cr.dia_corte =  @diaCorte15o30 and  cu.fecha_corte <= @fechaCorte
                    and cr.fk_id_estado = 1 and numero_cuota <> 0 and (cu.pago_total is null or cu.pago_total = 0))
                    or (cr.vigente = 1 and monto_capital_pagado < monto_desembolsado and cr.dia_corte =  @diaCorte15o30 and numero_cuota > 0 and pago_total = 1)
                    order by fk_id_credito


    select * into #cuotasViejas from #tempJoin as T1
    where not exists
    (
      select * from #tempJoin as T2
        where T1.fk_id_credito = T2.fk_id_credito
          and T1.fecha_corte > T2.fecha_corte
    )
```

Si es permitido realizar el proceso se realiza la consulta de todos los creditos vigentes que tengan el día de corte igual al correspondiente corte que se está intentando realizar junto a todas las cuotas de cada crédito encontrado que no han

sido pagadas completamente. Dicha consulta se guardará en una tabla temporal llamado #tempJoin.

Después de esto se realiza un filtrado de los datos de la consulta obteniendo por cada crédito encontrado la cuota más vieja vencida que tena relacionada y de igual manera esta consulta filtrada se guardara en una tabla temporal llamada #cuotasViejas.

```
BEGIN


    set @numeroDias =  DATEDIFF(day,@fecha_corte,@fechaCorte)

    if(@numeroDias > 9999)
    begin
        print @numeroDias

    end
    --print @fecha_corte
    --print @fechaCorte
    select  @idRangoDias = id_rangomora  from tbl_rangos_mora
        where @numeroDias >= dia_inicial
        and @numeroDias < dia_final or @numeroDias = dia_final
        and estado = 1

if @numero_cuota != 0
begin
    insert into tbl_saldos_corte
        values(@id_empresa, @fk_id_cupo, @fk_id_credito, @fechaCorte,
               @idRangoDias, @numeroDias, @saldo, @fechaEjecucion,null)
end
```

Por último con la tabla temporal creada llamada #cuotasViejas mediante un cursor en SQL se recorrerá registro por registro de dicha consulta y dentro de la lógica del cursor se hará una reclasificación de días de mora con respecto al rango encontrado en la tabla *tbl_rangos_mora* para por ultimo crear el registro en la tabla *tbl_saldos_corte*, dicho registro tendrá la información del crédito vigente al corte, el saldo que tiene y el número de días que este presenta con mora junto al id del cupo que tiene relacionado cada crédito.

- Ejecutar el script sp_debido_cobrar.sql que contiene la creación del procedimiento almacenado que permite establecer el valor pendiente de pago a cada cuota de los créditos vigentes a la fecha, incluyendo la mora respectiva si se da el caso.

```
Declare @ValidacionCorte Table(PermitirProceso bit,FechaCorte date, FechaEjecucion date)
    Insert @ValidacionCorte Exec sp_validar_corte_inicio_mes

select  @permitir = PermitirProceso from @ValidacionCorte
if @permitir = 1
begin...
end
```

El proceso de corte solo es permitido realizarlo en unas fechas determinadas por la lógica de negocio, para esto se implementó un procedimiento almacenado llamado sp_validar_corte_inicio_mes.sql que validara si la fecha en que se está ejecutando el proceso es la correcta, pues este devolverá el permiso y la fecha de corte válida para ejecutar el proceso.

```
SELECT cr.id_credito, emp.id_empresa, cr.fk_id_cupo, fk_id_tipo_credito,
    (cr.monto_desembolsado -cr.monto_capital_pagado) as saldo
    into #tempCreditos
    FROM tbl_creditos as cr join tbl_oficinas as ofi
    on CR.fk_id_oficina = ofi.id_oficina
    join tbl_marcas as marca on marca.id_marca = ofi.fk_id_marca
    join tbl_empresas as emp on emp.id_empresa = marca.fk_id_empresa
    where vigente = 1 and dia_corte =  @diaCorte15o30
    --and id_credito = 1042935

SELECT *  into #tempCuotas
    FROM   tbl_programacion_cuotas
    where  fecha_corte <= @fechaCorte
    AND(pago_total is null or pago_total = 0)

    select * into #tempJoin from #tempCreditos as cr
        inner join #tempCuotas as cu
        on cr.id_credito = cu.fk_id_credito order by fk_id_credito
```

El proceso continuara consultando los creditos vigentes a la fecha que correspondan al corte respectivo en que se está haciendo el proceso y por otro lado consultando en la programación de cuotas las cuotas que no hayan sido pagas que tenga como fecha de corte igual o menor a la fecha de corte en que se está realizando el proceso. Después con respecto a las dos consultas anteriores se creará una tabla temporal llamada #tempJoin que relaciona las dos consultas mencionadas.

```
CREATE TABLE #presupuestar (
    id_credito bigint,
    id_empresa bigint,
    fk_id_cupo bigint,
    fk_id_tipo_credito bigint,
    saldo money,
    id_programacioncuotas bigint,
    fk_id_credito bigint,
    fecha_corte DATE,
    fecha_maxima_pago DATE,
    numero_cuota INT,
    valor_capital money,
    valor_interes money,
    valor_iva_interes money,
    valor_aval money,
    valor_iva_aval money,
    valor_mora money,
    valor_iva_mora money,
    valor_capital_p money,
    valor_interes_p money,
    valor_iva_interes_p money,
    valor_aval_p money,
    valor_iva_aval_p money,
    valor_mora_p money,
    valor_iva_mora_p money,
    pago_total bit,
    id_origen bigint,
    numeroDias int,
    diasEntreCortes int,
    noMasMora bit,
    idRangoDias int,
    tasaMora decimal(6,4),
    impuestoMora decimal(6,4),
    valorMoraActual money,
    valorIvaMoraActual money,
    valorMoraTotal money,
    valorIvaMoraTotal money,
    valorCapitalInsert money,
    valorInteresInsert money,
    valorIvaInteresInsert money,
    valorAvalInsert money,
    valorIvaAvalInsert money,
    valorMoraTotalCorteAnterior money,
    valorIvaMoraTotalCorteAnterior money,

);
```

Se creará una tabla temporal que tendrá al final del proceso toda la información detallada de los valores que se deben presupuestar en dicho corte, todas las consultas realizadas en adelante serán para alimentar a esta tabla con sus respectivos datos.

```
INSERT INTO #presupuestar (
    id_credito, id_empresa, fk_id_cupo, fk_id_tipo_credito, saldo, id_programacioncuotas,
    fk_id_credito, fecha_corte, fecha_maxima_pago, numero_cuota, valor_capital,
    valor_interes, valor_iva_interes, valor_aval, valor_iva_aval, valor_mora,
    valor_iva_mora, valor_capital_p, valor_interes_p, valor_iva_interes_p, valor_aval_p,
    valor_iva_aval_p, valor_mora_p, valor_iva_mora_p, pago_total, id_origen,
    numeroDias, diasEntreCortes, noMasMora, idRangoDias, tasaMora, valorMoraActual,
    valorIvaMoraActual, valorMoraTotal, valorIvaMoraTotal, valorCapitalInsert,
    valorInteresInsert, valorAvalInsert, valorIvaAvalInsert
)
SELECT
    id_credito, id_empresa, fk_id_cupo, fk_id_tipo_credito, saldo, id_programacioncuotas,
    fk_id_credito, fecha_corte, fecha_maxima_pago, numero_cuota, valor_capital,
    valor_interes, valor_iva_interes, valor_aval, valor_iva_aval, valor_mora,
    valor_iva_mora, valor_capital_p, valor_interes_p, valor_iva_interes_p, valor_aval_p,
    valor_iva_aval_p, valor_mora_p, valor_iva_mora_p, pago_total, id_origen,
    0 as numeroDias, 0 as diasEntreCortes, 0 as noMasMora, 0 as idRangoDias, 0 as tasaMora,
    0 as valorMoraActual, 0 as valorIvaMoraActual, 0 as valorMoraTotal, 0 as valorIvaMoraTotal,
    0 as valorCapitalInsert, 0 as valorInteresInsert, 0 as valorAvalInsert, 0 as valorIvaAvalInsert
FROM #tempJoin;
```

En un primer momento a la tabla temporal creada #presupuestar se insertará todos los datos que contenga la tabla #tempoJoin que se tenía pues esta será la base donde continuara el proceso.

```
UPDATE #presupuestar SET numeroDias = CASE
    WHEN @diaCorte15o30 = 30 THEN DATEDIFF(day, EOMONTH(fecha_corte), @fechaUltimoDiaDelMes)
    ELSE DATEDIFF(day, fecha_corte, @fechaCorte)
    END;
```

Desde este punto se hará actualizaciones por cada registro creado en la tabla #presupuestar, en este caso se actualiza el campo *numeroDias* que contendrá los días de mora que cada cuota a presupuestar tiene a la fecha.

```
update #presupuestar SET #presupuestar.noMasMora = 0;
UPDATE #presupuestar SET #presupuestar.noMasMora = 1
            from #presupuestar  where numeroDias > 90;

update #presupuestar set idRangoDias = (select id_rangomora  from tbl_rangos_mora
                        where (numeroDias >= dia_inicial
                        and (numeroDias < dia_final or numeroDias = dia_final) )
                        and estado = 1)


UPDATE #presupuestar SET #presupuestar.tasaMora = c.valor_efectivo_mensual, #presupuestar.impuestoMora = c.impuesto
            from #presupuestar as a
            outer apply (
            select top(1) valor_efectivo_mensual, impuesto from tbl_tasas_mora where fk_id_tipo_credito = a.fk_id_tipo_credito and estado =1 order by fecha_fin desc
            ) as c
```

Luego se actualiza el campo *noMasMora* que será una bandera más adelante para saber que cuotas no se les debe presupuestar mora en el corte actual, la condición para que ocurra esta es si el numeroDias es mayor a 90. El valor establecido será un 1 para cuando se cumpla la condición.

Con respecto al número de días se actualiza también el campo *idRangoDias* que clasificara cada cuota por el rango de mora en que se encuentre.

En este punto también se encuentra la tasa de mora y el impuesto que maneja dicha tasa y se guardará por dicho valor por cada cuota registrada, este valor dependerá del tipo de crédito con el que está asociado dicha cuota.

```
update #presupuestar
    set valorCapitalInsert = valor_capital - valor_capital_p,
        valorInteresInsert = valor_interes - valor_interes_p,
        valorIvaInteresInsert = valor_iva_interes - valor_iva_interes_p,
        valorAvalInsert = valor_aval - valor_aval_p,
        valorIvaAvalInsert = valor_iva_aval - valor_iva_aval_p
where pago_total = 0

update #presupuestar
    set valorCapitalInsert = valor_capital,
        valorInteresInsert = valor_interes,
        valorIvaInteresInsert = valor_iva_interes,
        valorAvalInsert = valor_aval,
        valorIvaAvalInsert = valor_iva_aval
where pago_total is null
```

En esta parte del proceso se obtendrá el valor pendiente que tiene la cuota por los diferentes conceptos de capital, interés, aval y sus respectivos IVAs, esto se realiza con dos condiciones, si el estado *pago_total* de la cuota es 0 el valor que se debe presupuestar es la diferencia entre lo pagado con lo programado. Si el estado pago_total de la cuota es nulo significa que no hay ningún pago aplicado por lo tanto el valor que se tendrá que presupuestar será el valor completo de lo programado.

```
update #presupuestar set valorMoraActual =  round((((valorCapitalInsert + valorIvaAvalInsert + valorAvalInsert)*(tasaMora/100))/30)*@numeroDiasCalculoMoraActual,0) where noMasMora = 0
update #presupuestar set valorIvaMoraActual = round( valorMoraActual *(impuestoMora/100),0) where noMasMora = 0

update #presupuestar set valorMoraActual =  0 where fecha_corte = @fechaCorte
update #presupuestar set valorIvaMoraActual = 0  where fecha_corte = @fechaCorte
```

Luego continuara el proceso de calcular la mora correspondiente al nuevo corte que se está realizando, para esto se utiliza la formula 1 que hace el cálculo de la mora a las cuotas que anteriormente se dejaron marcadas con la bandera de noMasMora igual a 0, también se garantiza que para las cuotas que apenas se presupuestan por primera vez en este corte no deben tener ningún valor en este ítem y se garantiza el 0 en dichas cuotas.

$$Mora = \left(\frac{\left((Capital + Aval + IvaAval) \cdot \left(\frac{tasaMora}{100} \right) \right)}{30} \right) \cdot diasEnMora \qquad (1)$$

```
select
    case
        when a.noMasMora = 0  and c.pago_total is null then (isnull(b.valor_mora_total,0) - isnull(c.valor_mora_p,0))
        when a.noMasMora = 0  and c.pago_total = 0 then (isnull(c.valor_mora,0))
        when a.noMasMora = 1  and c.pago_total = 0 then (isnull(c.valor_mora,0) - ISNULL(c.valor_mora_p,0))
        when a.noMasMora = 1  and c.pago_total is null then (isnull(c.valor_mora,0))

    end
as valor_mora_total,
    case
        when a.noMasMora = 0  and c.pago_total is null then (isnull(b.valor_iva_mora_total,0) - isnull(c.valor_iva_mora_p,0))
        when a.noMasMora = 0  and c.pago_total = 0 then (isnull(c.valor_iva_mora,0))
        when a.noMasMora = 1  and c.pago_total = 0 then (isnull(c.valor_iva_mora,0) - ISNULL(c.valor_iva_mora,0))
        when a.noMasMora = 1  and c.pago_total is null then (isnull(c.valor_iva_mora,0))

    end
as valor_iva_mora_total,
fk_id_cuota
into #moraAnterior from #presupuestar as a
left join tbl_presupuesto_cuotas as b on
b.fk_id_cuota = a.id_programacioncuotas and  b.fecha_corte = @fechaUltimoCorte
join tbl_programacion_cuotas as c on c.id_programacioncuotas = b.fk_id_cuota
```

En este proceso se calculará lo que será el valor de mora total por cada cuota, el cálculo pasado categorizado como mora actual es apenas el valor nuevo o adicional que se le debe de cobrar a un cliente en la cuota, pero como las cuotas podrían venir desde antes con un valor de mora se debe sumar esta mora actual nueva a ese valor total. En la imagen anterior se establecen unas condiciones para conocer dicho valor y guardarse en una tabla temporal llamada #moraAnterior. Esto se hace para todas las cuotas que se encuentran en el proceso de presupuesto en simultaneo que cumplan con las condiciones dadas.

```
UPDATE #presupuestar
SET  #presupuestar.valorMoraTotal = round(isnull(#presupuestar.valorMoraActual,0) +  isnull(#moraAnterior.valor_mora_total,0),0),
     #presupuestar.valorIvaMoraTotal = round(isnull(#presupuestar.valorIvaMoraActual,0) + isnull(#moraAnterior.valor_iva_mora_total,0),0)
FROM #presupuestar
JOIN #moraAnterior ON #moraAnterior.fk_id_cuota = #presupuestar.id_programacioncuotas

update tbl_programacion_cuotas
set tbl_programacion_cuotas.valor_mora = #presupuestar.valorMoraTotal,
    tbl_programacion_cuotas.valor_iva_mora = #presupuestar.valorIvaMoraTotal
from tbl_programacion_cuotas
join #presupuestar on #presupuestar.id_programacioncuotas = tbl_programacion_cuotas.id_programacioncuotas
WHERE #presupuestar.valorMoraTotal > 0
```

Cuando se tenga la tabla temporal #moraAnterior con la información de la mora total con la que quedara cada cuota se actualizara la tabla de tbl_programacion_cuotas con el nuevo valor de la mora para el corte. Con este último paso ya se tiene toda la información necesaria para crear el presupuesto del corte y así finalizar el debido cobro.

```
INSERT INTO [dbo].[tbl_presupuesto_cuotas]
           ([fk_id_empresa]
           ,[fk_id_cupo]
           ,[fk_id_tipo_credito]
           ,[fk_id_credito]
           ,[fk_id_cuota]
           ,[fecha_corte]
           ,[fecha_real_cuota]
           ,[fk_id_rango_mora]
           ,[dias_mora]
           ,[valor_capital]
           ,[valor_interes]
           ,[valor_iva_interes]
           ,[valor_aval]
           ,[valor_iva_aval]
           ,[valor_mora_actual]
           ,[valor_iva_mora_actual]
           ,[valor_mora_total]
           ,[valor_iva_mora_total]
           ,[id_origen])
    select id_empresa as fk_id_empresa, fk_id_cupo, fk_id_tipo_credito, fk_id_credito, id_programacioncuotas as fk_id_cuota,
    @fechaCorte as fecha_corte, fecha_corte as fecha_real_cuota, idRangoDias as fk_id_rango_mora, numeroDias as dias_mora,
    valorCapitalInsert as valor_capital, valorInteresInsert as valor_interes, valorIvaInteresInsert as valor_iva_interes,
    valorAvalInsert as valor_aval, valorIvaAvalInsert as valor_iva_aval, valorMoraActual as valor_mora_actual, valorIvaMoraActual as valor_iva_mora_actual,
    valorMoraTotal  as valor_mora_total, valorIvaMoraTotal as valor_iva_mora_total, null as id_origen
    from #presupuestar
```

El proceso finaliza haciendo la inserción de los datos que se crearon temporalmente en la tabla #presupuestar en la tabla tbl_presupuesto_cuotas. Finalizando así el proceso de debido cobro.

- Ejecutar el script sp_crear_valores_facturas.sql que contiene la creación del procedimiento almacenado que realiza la inserción de datos de las facturas POS y/o electronicas en sus respectivas tablas.

```
Declare @ValidacionCorte Table(PermitirProceso bit,FechaCorte date, FechaEjecucion date)
    Insert @ValidacionCorte Exec sp_validar_corte_inicio_mes

select  @permitir = PermitirProceso from @ValidacionCorte
if @permitir = 1
begin...
end
```

El proceso de corte solo es permitido realizarlo en unas fechas determinadas por la lógica de negocio, para esto se implementó un procedimiento almacenado llamado sp_validar_corte_inicio_mes.sql que validara si la fecha en que se está ejecutando el proceso es la correcta, pues este devolverá el permiso y la fecha de corte válida para ejecutar el proceso.

El proceso de facturacion depende única y exclusivamente de los valores presupuestados en el corte, por lo tanto, este proceso debe garantizarse antes de iniciar con la facturacion. Los valores presupuestados son los valores que se le van a facturar al cliente y los que deberían ser cobrados posterior a esta fecha. Si hay valores se ya fueron pagos, pero nunca se presupuestaron se deben facturar como un valor anticipado.

```
CREATE TABLE #facturacion(
    [id_cupo] [bigint],
    [id_cliente] [bigint]  ,
    [valor_interes] [money]  ,
    [valor_iva_interes] [money]  ,
    [valor_aval] [money]  ,
    [valor_iva_aval] [money]  ,
    [valor_mora] [money]  ,
    [valor_iva_mora] [money]  ,
    [valor_interes_ant] [money]  ,
    [valor_iva_interes_ant] [money]  ,
    [valor_aval_ant] [money]  ,
    [valor_iva_aval_ant] [money]  ,
    [valor_mora_ant] [money]  ,
    [valor_iva_mora_ant] [money])
```

Para agrupar los valores que se facturarán por cliente, se creará una tabla temporal llamada #facturacion que tendrá al final todos los registros con su detalle agrupados por cupo.

```
Select fk_id_cupo
into #CuposQueNoSeDebenFacturar
from tbl_presupuesto_cuotas
where fecha_corte = @fechaCorte and dias_mora >= 90
group by fk_id_cupo

--Actualizar en la programacion de cuotas un 0 en el campo cuota_no_facturadas
update tbl_programacion_cuotas set cuota_no_facturada = 0
where id_programacioncuotas in (
    Select fk_id_cuota
    from tbl_presupuesto_cuotas
    where fecha_corte = @fechaCorte and
    fk_id_cupo in
        (Select fk_id_cupo
                        from tbl_presupuesto_cuotas
                        where fecha_corte = @fechaCorte  and dias_mora >= 90
                        group by fk_id_cupo, fecha_corte
        ) and valor_mora_actual > 0
)

--Actualizar en la programacion de cuotas un 1 en el campo cuota_no_facturadas
update tbl_programacion_cuotas set cuota_no_facturada = 1
    where id_programacioncuotas in (
    Select fk_id_cuota
    from tbl_presupuesto_cuotas
    where fecha_corte = @fechaCorte and
    fk_id_cupo in (Select fk_id_cupo
                        from tbl_presupuesto_cuotas
                        where fecha_corte = @fechaCorte  and dias_mora >= 90
                        group by fk_id_cupo, fecha_corte
                ) and fecha_real_cuota = @fechaCorte
    )
```

Al principio del proceso se consulta de todo el presupuesto creado al corte cuales de los cupos tienen una mora mayor a 90 días y se agrupa esta información por cupo guardándolo en una tabla temporal llamada #CuposQueNoSeDebenFacturar.

Con respecto a esto, por cupo existirán nuevas cuotas al corte que se presupuestaron y se les cobrara a los clientes en caso de que paguen y otras cuotas que no pertenecen al corte si no que son cuotas que cortaron meses anteriores y en ese momento fueron facturados. El primer caso se considerará que la cuota nunca fue facturada por lo tanto en la bandera que se tiene en la cuota llamada *cuota_no_facturada* se marcara con un 1 para futuros procesos, El segundo caso se considera que la cuota ya fue facturada en algún punto, pero hubo valores nuevos que se dejaron de facturar por lo que al campo *cuota_no_facturada* se marcara con un valor de 0 indicando que dicha cuota ha sido facturada parcialmente.

```
-- insertar los valores nuevos de interes del corte actual (fechaRealCorte == fechaCorte)
INSERT INTO #facturacion (id_cupo, id_cliente, valor_interes, valor_iva_interes, valor_aval, valor_iva_aval, valor_mora, valor_iva_mora, valor_interes_ant,
valor_iva_interes_ant, valor_aval_ant, valor_iva_aval_ant, valor_mora_ant, valor_iva_mora_ant)
    SELECT  Pre.fk_id_cupo AS id_cupo,
        Cu.fk_id_cliente AS id_cliente,
        SUM(Pre.valor_interes) as interes,
        SUM(Pre.valor_iva_interes) as ivaInteres,
        SUM(Pre.valor_aval) as aval,
        SUM(Pre.valor_iva_aval) as ivaAval,
        0 as mora,
        0 as ivaMora,
        0 as interesAnt,
        0 as ivaInteresAnt,
        0 as moraAnt,
        0 as ivaMoraAnt,
        0 as avalAnt,
        0 as ivaAvalAnt
        FROM tbl_presupuesto_cuotas AS Pre
    JOIN tbl_cupos AS Cu ON Pre.fk_id_cupo = Cu.id_cupo
    LEFT JOIN #CuposQueNoSeDebenFacturar AS NoFac ON Cu.id_cupo = NoFac.fk_id_cupo
    WHERE Pre.fecha_real_cuota = @fechaCorte  AND NoFac.fk_id_cupo is null --fechaCorte
          GROUP BY Pre.fk_id_cupo, Cu.fk_id_cliente
        ORDER BY Pre.fk_id_cupo DESC
```

El proceso de facturacion continúa conociendo que valores nuevos fueron presupuestados a la fecha de corte, para esto se debe cumplir la condición que la cuota tenga programada únicamente la fecha corte que se está ejecutando. Esta condición garantiza que se facturen valores que antes ya se habían facturado. Los valores nuevos que podrán ser facturados pertenece a los conceptos de interés, aval e IVA de dichas cuotas. Aquí se hará uso de la tabla temporal #CuposQueNoSeDebenFacturar pues los cupos que estén registrados en dicha tabla deberán ser ignoradas y no podrán ser facturadas para el corte.

```
-- insertar los valores nuevos de mora de cortes pasados (fechaCorte == fechaCorte)
INSERT INTO #facturacion (id_cupo, id_cliente, valor_interes, valor_iva_interes, valor_aval, valor_iva_aval, valor_mora, valor_iva_mora, valor_interes_ant,
valor_iva_interes_ant, valor_aval_ant, valor_iva_aval_ant, valor_mora_ant, valor_iva_mora_ant)
    SELECT  Pre.fk_id_cupo AS id_cupo,
        Cu.fk_id_cliente AS id_cliente,
        0 as interes,
        0  as ivaInteres,
        0 as aval,
        0 as ivaAval,
        SUM(Pre.valor_mora_actual) as mora,
        SUM(Pre.valor_iva_mora_actual) as ivaMora,
        0 as interesAnt,
        0 as ivaInteresAnt,
        0 as moraAnt,
        0 as ivaMoraAnt,
        0 as avalAnt,
        0 as ivaAvalAnt
        FROM tbl_presupuesto_cuotas AS Pre
    JOIN tbl_cupos AS Cu ON Pre.fk_id_cupo = Cu.id_cupo
    LEFT JOIN #CuposQueNoSeDebenFacturar AS NoFac ON Cu.id_cupo = NoFac.fk_id_cupo
    WHERE Pre.fecha_corte = @fechaCorte AND NoFac.fk_id_cupo is null --fechaCorte
          GROUP BY Pre.fk_id_cupo, Cu.fk_id_cliente
        ORDER BY Pre.fk_id_cupo DESC
```

Luego continua el proceso de facturar la nueva mora presupuestada en el corte, para esto solo se tiene en cuenta el ítem *valor_mora_actual* del presupuesto que discrimina ya aquí la nueva mora y se pone como condición que la fecha corte cuando se realizó el presupuesto sea la misma fecha del proceso. De la misma manera se excluye de la facturacion los cupos registrados en #CuposQueNoSeDebenFacrturar.

Teniendo ya registrados los nuevos valores que se deben facturar se continúa conociendo que valores se deben facturar como anticipados, hay varios procesos en credi10 que hace que se tenga valores anticipados que facturar.

```
--DESDE AQUI SE CALCULAN LOS VALORES ANTICIPADOS
--Insertar los valores de interes anticipados por las cuotas 0 creadas desde el ultimo corte    ****esto depende de la fecha de migracion
INSERT INTO #facturacion (id_cupo, id_cliente, valor_interes, valor_iva_interes, valor_aval, valor_iva_aval, valor_mora, valor_iva_mora, valor_interes_ant,
valor_iva_interes_ant, valor_aval_ant, valor_iva_aval_ant, valor_mora_ant, valor_iva_mora_ant)
SELECT  cu.id_cupo AS id_cupo,
        Cu.fk_id_cliente AS id_cliente,
        0 as interes,
        0  as ivaInteres,
        0 as aval,
        0 as ivaAval,
        0 as mora,
        0 as ivaMora,
        SUM(c.valor_interes_p) as interesAnt,
        SUM(c.valor_iva_interes_p) as ivaInteresAnt,
        0 as moraAnt,
        0 as ivaMoraAnt,
        0 as avalAnt,
        0 as ivaAvalAnt
        FROM tbl_programacion_cuotas AS c
    LEFT JOIN tbl_creditos AS Cr ON c.fk_id_credito = Cr.id_credito
    LEFT JOIN tbl_cupos AS cu on Cr.fk_id_cupo = cu.id_cupo
    LEFT JOIN tbl_presupuesto_cuotas as p on cu.id_cupo = p.fk_id_cupo and p.fecha_corte = @FechaCorteAnterior
    WHERE (CAST(c.fecha_corte AS DATE) > IIF(@diaCorte = 15, @FechaCorteAnterior, EOMONTH(@FechaCorteAnterior, 0)) AND
          CAST(c.fecha_corte AS DATE) <= GETDATE() and numero_cuota = 0 and valor_interes_p > 0)  AND cu.dia_corte = @diaCorte AND p.id_presupuestocuota IS NULL
            AND (c.cuota_no_facturada is null OR c.cuota_no_facturada = 0)
            GROUP BY cu.id_cupo,cr.id_credito, Cu.fk_id_cliente
            ORDER BY id_credito
```

El primer caso se da con los clientes que tienen creditos refinanciados y aplican un pago antes de su primer corte. En las refinanciaciones el valor de la primera cuota programada se debe garantizar que se pague por completo, si esta no ha pasado por corte sus valores de interés se cobraran y al no haber sido presupuestados se tratan como un valor anticipado que debe ser facturado en el corte que se está realizando.

Para los creditos normales, si un cliente realiza un abono antes de su primer corte en este caso si se cobra los intereses corrientes transcurridos hasta la fecha y dicho pago quedara asentado en una cuota 0. Esto también será considerado como un valor anticipado de interés que deberá ser facturado en el corte.

```
   --CALCULO DE LOS VALORES ANTICIPADOS QUE NO SE FACTURARON EL CORTE PASADO
 SELECT fk_id_cupo, SUM(valor_interes) AS interes,
       SUM(valor_iva_interes) AS ivaInteres,
       SUM(valor_mora_total) AS mora,
       SUM(valor_iva_mora_total) AS ivaMora
   INTO #Facturas
       FROM tbl_presupuesto_cuotas
WHERE fecha_corte = @FechaCorteAnterior
GROUP BY fk_id_cupo


 SELECT C.fk_id_cupo, C.fk_id_cliente, MIN(P.numero_cuota) as numeroCuota,
       SUM(CD.valor_interes) AS interes, SUM(CD.valor_iva_interes) AS ivaInteres, SUM(CD.valor_mora) AS mora, SUM(CD.valor_iva_mora) AS ivaMora
   INTO #RC
FROM tbl_comprobantes AS C
JOIN tbl_comprobante_detalles AS CD ON C.id_comprobante = CD.fk_id_comprobante
JOIN tbl_programacion_cuotas AS P ON CD.fk_id_cuota = P.id_programacioncuotas
JOIN tbl_cupos AS cu ON cu.id_cupo = C.fk_id_cupo
WHERE CAST(C.fecha AS DATE) > IIF(@diaCorte = 15, @FechaCorteAnterior, EOMONTH(@FechaCorteAnterior, 0)) AND
CAST(C.fecha AS DATE) <= CAST(GETDATE() AS DATE) AND
(fk_id_documento_transaccional in (Select fk_id_doc_recibo_caja from tbl_documentos_clasificacion) OR
fk_id_documento_transaccional in (Select fk_id_doc_nota_credito_refinanciacion from tbl_documentos_clasificacion) OR
fk_id_documento_transaccional in (Select fk_id_doc_nota_credito from tbl_documentos_clasificacion)
) and cu.dia_corte = @diaCorte  AND (P.cuota_no_facturada is null OR P.cuota_no_facturada = 0)
GROUP BY C.fk_id_cupo, C.fk_id_cliente, C.fecha
order by fecha


Select fk_id_cupo, fk_id_cliente, MIN(numeroCuota) as numeroCuota, SUM(interes) as interes, SUM(ivaInteres) as ivaInteres,
       SUM(mora) as mora, SUM(ivaMora) as ivaMora
into #RCGroup from #RC group by fk_id_cupo, fk_id_cliente
```

Existen otros valores anticipados que son por abonos extras que realizan durante el mes y las cuotas donde van a parar dichos abonos ya han pasado por corte. Para encontrar dichos valores que un cliente pago de manera anticipada se busca el

presupuesto anterior y el detalle de los pagos realizados durante el último mes. Para esto se crean dos tablas temporales que condesaran dicha información.

```
--Insertar los valores de interes y mora anticipados que no se facturaron el corte pasado
INSERT INTO #facturacion (id_cupo, id_cliente, valor_interes, valor_iva_interes, valor_aval, valor_iva_aval, valor_mora, valor_iva_mora, valor_interes_ant,
valor_iva_interes_ant, valor_aval_ant, valor_iva_aval_ant, valor_mora_ant, valor_iva_mora_ant)
Select  F.fk_id_cupo, RC.fk_id_cliente,  0 as interes,0  as ivaInteres,0 as aval,  0 as ivaAval, 0 as mora, 0 as ivaMora,
IIF(SUM(RC.interes) - SUM(F.interes) < 0, 0, SUM(RC.interes) - SUM(F.interes)) AS interesAnticipado,
IIF(SUM(RC.ivaInteres) - SUM(F.ivaInteres) < 0, 0, SUM(RC.ivaInteres) - SUM(F.ivaInteres)) AS ivaIntAnticipado, 0 as avalAnt, 0 as ivaAvalAnt,
IIF(SUM(RC.mora) - SUM(F.mora) <0, 0, SUM(RC.mora) - SUM(F.mora)) as moraAnticipado,
IIF(SUM(RC.ivaMora) - SUM(F.ivaMora) < 0, 0, SUM(RC.ivaMora) - SUM(F.ivaMora)) as ivaMoraAnticipado
  FROM #Facturas AS F
 JOIN #RCGroup AS RC ON F.fk_id_cupo = RC.fk_id_cupo
 WHERE RC.interes > F.interes OR RC.mora > F.mora
 GROUP BY F.fk_id_cupo, RC.fk_id_cliente
 ORDER BY F.fk_id_cupo

       INSERT INTO #facturacion (id_cupo, id_cliente, valor_interes, valor_iva_interes, valor_aval, valor_iva_aval, valor_mora, valor_iva_mora, valor_interes_ant,
valor_iva_interes_ant, valor_aval_ant, valor_iva_aval_ant, valor_mora_ant, valor_iva_mora_ant)
SELECT  RC.fk_id_cupo,  RC.fk_id_cliente, 0 as interes, 0  as ivaInteres, 0 as aval,  0 as ivaAval, 0 as mora,  0 as ivaMora,
IIF(SUM(RC.interes) < 0, 0, SUM(RC.interes)) AS interesAnticipado,
IIF(SUM(RC.ivaInteres) < 0, 0, SUM(RC.ivaInteres)) AS ivaIntAnticipado,  0 as avalAnt,  0 as ivaAvalAnt,
IIF(SUM(RC.mora) <0, 0, SUM(RC.mora)) as moraAnticipado,
IIF(SUM(RC.ivaMora) < 0, 0, SUM(RC.ivaMora)) as ivaMoraAnticipado
FROM #RCGroup
as RC
LEFT JOIN #Facturas AS F ON rc.fk_id_cupo = F.fk_id_cupo
WHERE F.fk_id_cupo IS NULL and numeroCuota <> 0 GROUP BY rc.fk_id_cupo, RC.fk_id_cliente
 order by rc.fk_id_cupo;


    DROP TABLE #RCGroup
    DROP TABLE #Facturas
    DROP TABLE #RC
```

Para encontrar los valores anticipados se registrará la diferencia que exista entre lo pagado por el cliente por el valor real presupuestado en el corte anterior. En otras palabras, lo que hace esto es que, si el cliente en el corte pasado se le facturo 1 peso de interés y durante el mes le cobre 2 pesos de interés por el abono demás realizado, para este corte ese 1 peso de más entrara como valor anticipado en la factura que se le realice.

```
-------------------------------------------------------------------------------------------------------------------
--------------------    nuevo proceso para facturar valores cobrados pero no facturados en cortes anteriores  ------------------

--Cuotas completas que no se facturaron

INSERT INTO #facturacion (id_cupo, id_cliente, valor_interes, valor_iva_interes, valor_aval, valor_iva_aval, valor_mora, valor_iva_mora, valor_interes_ant,
       valor_iva_interes_ant, valor_aval_ant, valor_iva_aval_ant, valor_mora_ant, valor_iva_mora_ant)
Select a.fk_id_cupo, a.fk_id_cliente,
0,0,0,0,0,0,
SUM(b.valor_interes) interes, SUM(b.valor_iva_interes) as ivaInteres,
SUM(b.valor_aval) as aval, SUM(b.valor_iva_aval) as ivaAval, SUM(b.valor_mora) as mora, SUM(b.valor_iva_mora) as ivaMora
from tbl_comprobantes as a
join tbl_comprobante_detalles as b on a.id_comprobante = b.fk_id_comprobante
join tbl_presupuesto_cuotas as c on b.fk_id_cuota = c.fk_id_cuota and fecha_corte = @FechaCorteAnterior
where b.fk_id_cuota in (Select id_programacioncuotas from tbl_programacion_cuotas where cuota_no_facturada = 1 and pago_total = 1)
and CAST(fecha as date) > cast(EOMONTH(@FechaCorteAnterior,0) as date)
group by a.fk_id_cupo,a.fk_id_cliente

--Como se van a facturar las cuotas se actualiza el campo cuota_no_facturada en null
update tbl_programacion_cuotas
set cuota_no_facturada = null
where cuota_no_facturada = 1 and pago_total = 1
```

Otro valor anticipado que se facturara es con el mismo proceso que se realizó al principio en encontrar los cupos que no se les va a facturar en cada corte. A estos cupos que se les dejo de facturar y quedad marcados se les volverá a facturar en el corte más próximo después de que dicho cliente haya realizado el pago completo de la cuota. De esta manera será fácil identificar cuáles son las cuotas que no se les facturo y que ya han sido pagas.

Si una cuota al corte que se está evaluando cumple con la condición de que el campo cuota_no_facturada es igual a 1 y el campo pago_total es igual a 1, los valores pagados serán facturados como anticipados para el corte actual.

Y el estado de la cuota en el campo *cuota_no_factura* pasara actualizarse con un valor nulo para finalizar allí el proceso de facturacion y sacar del mapa por completo de la lógica a dicha cuota.

```
--Cuotas que estan facturadas parcialmente

INSERT INTO #facturacion (id_cupo, id_cliente, valor_interes, valor_iva_interes, valor_aval, valor_iva_aval, valor_mora, valor_iva_mora, valor_interes_ant,
       valor_iva_interes_ant, valor_aval_ant, valor_iva_aval_ant, valor_mora_ant, valor_iva_mora_ant)
Select a.fk_id_cupo, a.fk_id_cliente,
0,0,0,0,0,0,0,0,0,
SUM(MoraPres.valor_mora_actual) as mora, SUM(IvaMoraPres.valor_iva_mora_actual) as ivaMora
from tbl_comprobantes as a
join tbl_comprobante_detalles as b on a.id_comprobante = b.fk_id_comprobante
outer apply(
    Select distinct(valor_mora_actual)
from  tbl_presupuesto_cuotas as c where b.fk_id_cuota = c.fk_id_cuota and fecha_corte = @FechaCorteAnterior
) as MoraPres
outer apply(
    Select distinct(valor_iva_mora_actual)
from  tbl_presupuesto_cuotas as c where b.fk_id_cuota = c.fk_id_cuota and fecha_corte = @FechaCorteAnterior
) as IvaMoraPres
where b.fk_id_cuota in (Select id_programacioncuotas from tbl_programacion_cuotas where cuota_no_facturada = 0 and pago_total = 1)
and CAST(fecha as date) > cast(EOMONTH(@FechaCorteAnterior,0) as date)
group by a.fk_id_cupo, a.fk_id_cliente

--Como se va a facturar el saldo de mora que no facturo en el corte pasado se actualiza el campo cuota_no_facturada en null
update tbl_programacion_cuotas
set cuota_no_facturada = null
where cuota_no_facturada = 0 and pago_total = 1
```

Si es el caso que la cuota esta facturada parcialmente y ya ha sigo paga por completa, los valores que deberán ser facturadas como anticipados será el valor de mora que se le presupuesto en el último corte. Al final también será actualizada el campo cuota_no_factura por un valor nulo para indicar que ya los valores de dicha cuota fueron pagados y facturados.

```
SELECT  id_cupo AS cupo,
       id_cliente AS cliente,
       (SUM(valor_interes) + SUM(valor_iva_interes) + SUM(valor_aval) + SUM(valor_iva_aval) +
       SUM(valor_mora) + SUM(valor_iva_mora) + SUM(valor_interes_ant) + SUM(valor_iva_interes_ant) +
       SUM(valor_mora_ant) + SUM(valor_iva_mora_ant) + SUM(valor_aval_ant) + SUM(valor_iva_aval_ant)) as valor_total,
       SUM(valor_interes) as interes,
       SUM(valor_iva_interes) as ivaInteres,
       SUM(valor_aval) as aval,
       SUM(valor_iva_aval) as ivaAval,
       SUM(valor_mora) as mora,
       SUM(valor_iva_mora) as ivaMora,
       SUM(valor_interes_ant) as interesAnt,
       SUM(valor_iva_interes_ant) as ivaInteresAnt,
       SUM(valor_mora_ant) as moraAnt,
       SUM(valor_iva_mora_ant) as ivaMoraAnt,
       SUM(valor_aval_ant) as avalAnt,
       SUM(valor_iva_aval_ant) as ivaAvalAnt
    INTO #tempFacturas
    from #facturacion
        GROUP BY id_cupo, id_cliente
        order by id_cupo
```

Ya en este punto han sido registrados todos los valores que se deberán facturar para el corte entre los diferentes conceptos que existen, lo que se hará es agrupar dicha información y valores para que quede un registro de factura por cliente. Los datos agrupados serán guardados en la tabla temporal #tempFacturas

```
IF @valorTotal < @totalUVT OR @encendidoFacturacionElectronica = 0
    BEGIN
        IF @valorTotal <> 0
        --Registro una factura POS si el valorTotal es menor al totalUVT o si esta apagado la facturacionElectronica
            insert into tbl_facturaciones
                values(@idCupo, @idCliente, null, @FechaFacturacion, @FechaFacturacion,
                (@valorInteres + round(@valorInteres * 0.19, 0) + @valorAval + @valorIvaAval +
        @valorMora + @valorIvaMora + @valorinteresAnt + round(@valorinteresAnt * 0.19, 0) +
        @valorMoraAnt + round(@valorMoraAnt * 0.19, 0) + @valorAvalAnt + round(@valorAvalAnt * 0.19, 0))

        , @valorInteres, round(@valorInteres * 0.19, 0), @valorAval, @valorIvaAval,
                @valorMora, @valorIvaMora, @valorinteresAnt, round(@valorinteresAnt * 0.19, 0),
                @valorMoraAnt, round(@valorMoraAnt * 0.19, 0), @valorAvalAnt, round(@valorAvalAnt * 0.19, 0),0,
                replace(newid(), '-', ''), null)
    END
ELSE
    BEGIN
        IF @valorTotal <> 0
            INSERT INTO tbl_facturacion_electronica
            (fk_id_cupo, fk_id_cliente, fk_id_comprobante_factura, fk_id_comprobante_nota, fecha_facturacion,
            fecha_corte, valor_total, valor_interes, valor_iva_interes, valor_aval, valor_iva_aval,
            valor_mora, valor_iva_mora, valor_anticipado_interes, valor_anticipado_iva_interes,
            valor_anticipado_mora, valor_anticipado_iva_mora, valor_anticipado_aval, valor_anticipado_iva_aval,
            correo_enviado, cliente_enviado_siesa, cliente_xml, cliente_respuesta, cliente_creado,
            factura_enviada_siesa, factura_xml, factura_respuesta, factura_creada, nota_enviada_siesa, nota_xml,
            nota_respuesta, nota_creada, fk_id_doc_factura_electronica)
            VALUES(@idCupo, @idCliente, null, null, @FechaFacturacion, @FechaFacturacion,
            (@valorInteres + round(@valorInteres * 0.19, 0) + @valorAval + @valorIvaAval +
            @valorMora + @valorIvaMora + @valorinteresAnt + round(@valorinteresAnt * 0.19, 0) +
            @valorMoraAnt + round(@valorMoraAnt * 0.19, 0) + @valorAvalAnt + round(@valorAvalAnt * 0.19, 0)),
            @valorInteres, round(@valorInteres * 0.19, 0), @valorAval, @valorIvaAval,
            @valorMora, @valorIvaMora, @valorinteresAnt, round(@valorinteresAnt * 0.19, 0),
            @valorMoraAnt, round(@valorMoraAnt * 0.19, 0), @valorAvalAnt, round(@valorAvalAnt * 0.19, 0), 0, 0,
            null, null, 0, 0, null, null, 0, 0, null, null, 0, null);
    END
```

Por último, se tendrá un cursor que va a recorrer la tabla #tempFacturas y hará el registro de la factura correspondiente, siempre y cuando el valor de este sea mayor a 0. Para crear una factura POS se tendrá que realizar el registro en la tabla tbl_facturaciones y si se debe crear una factura electrónica se deberá crear un registro en la tabla tbl_facturacion_electronica. Esta distinción entre facturas será dada por parámetros dados como, si esta encendida la facturacion electrónica, y si lo está cuanto es el valor mínimo dado por la cantidad de UVT parametrizado. Dicho valor dará la condición para crear una u otra factura.

- Ejecutar el script sp_crear_comprobantes_facturas.sql que contiene la creación del procedimiento almacenado que realiza los comprobantes (documento C10) de las facturas POS creadas en el corte.

```
--Obtengo el id de la empresa
SELECT @IdEmpresaVarChar = valor FROM tbl_parametros WHERE key_ = 'IdEmpresaFacturacion'
SET @IdEmpresa = CAST(@IdEmpresaVarChar AS INT)
--Obtengo el documento transaccional para facturacion
SELECT top(1) @IdDocumentoClasificacion = fk_id_doc_facturas FROM tbl_documentos_clasificacion
--Obtengo el consecutivo correspondiente
SELECT @ConsecutivoActual = actual FROM tbl_consecutivos
    where fk_id_empresa = @IdEmpresa
            AND fk_id_documentos_transaccionales = @IdDocumentoClasificacion
            AND GETDATE() BETWEEN vigencia_fecha_inicio AND vigencia_fecha_fin
--Obtengo los valores de las facturas
SELECT id_facturacion AS factura
      ,fk_id_cupo AS cupo
      ,fk_id_cliente AS cliente
      ,valor_total AS valorTotal
      ,(valor_interes + valor_anticipado_interes) AS interes
      ,(valor_iva_interes + valor_anticipado_iva_interes) AS ivaInteres
      ,valor_aval AS aval
      ,valor_iva_aval AS ivaAval
      ,(valor_mora + valor_anticipado_mora) AS mora
      ,(valor_iva_mora + valor_anticipado_iva_mora) AS ivaMora
    INTO #tempFacturas
  FROM tbl_facturaciones
    WHERE fecha_corte = @FechaRealFactura
    AND fk_id_comprobante IS NULL
    ORDER BY fk_id_cupo ASC
--Empiezo el cursor
```

Para crear los comprobantes C10 correspondientes a las facturas POS del corte se busca en los parámetros el documento, la empresa que factura y el consecutivo actual desde donde se debe continuar la creación de los documentos.

Después se obtiene los valores registrados en la tabla *tbl_facturaciones* al corte en que se está haciendo el proceso.

```
    --Aqui realizo los inserts en comprobantes y detalle de comprobantes
    SET @ConsecutivoActual = @ConsecutivoActual + 1;

    INSERT INTO tbl_comprobantes (fk_id_documento_transaccional, consecutivo, fecha,
    fk_id_cliente, fk_id_cupo, valor_total, fk_id_empresa)
        VALUES (@IdDocumentoClasificacion, @ConsecutivoActual, @FechaRealFactura,
            @idCliente, @idCupo, @valorTotal, @IdEmpresa);

    SELECT @IdUltimoComprobante =  SCOPE_IDENTITY();

    INSERT INTO tbl_comprobante_detalles (fk_id_comprobante, valor_debito, valor_credito,
    valor_capital, valor_interes, valor_iva_interes, valor_aval, valor_iva_aval, valor_mora,
    valor_iva_mora)
        VALUES (@IdUltimoComprobante, @valorTotal, 0, 0, @valorInteres, @valorIvaInteres,
        @valorAval, @valorIvaAval, @valorMora, @valorIvaMora)

    --Actualizo el campo de comprobantes en la tabla de facturacion
    UPDATE tbl_facturaciones
    SET fk_id_comprobante = @IdUltimoComprobante
    WHERE id_facturacion = @IdFactura
```

Después se entra a un cursor para crear el comprobante y el detalle respectivo por cada factura registrada, aumentando en cada comprobante el número de consecutivo con el que se está creando el documento.

```
UPDATE tbl_consecutivos
SET actual = @ConsecutivoActual
WHERE fk_id_empresa =  @IdEmpresa
                AND fk_id_documentos_transaccionales = @IdDocumentoClasificacion
                AND GETDATE() BETWEEN vigencia_fecha_inicio AND vigencia_fecha_fin
```

Cuando finalice el proceso se deberá actualizar en la tabla *tbl_consecutivos* el valor actual con el que quedo creado el último documento (C10) creado en el proceso.

- Ejecutar el script sp_crear_comprobantes_facturas_electronicas.sql que contiene la creación del procedimiento almacenado que realiza los comprobantes (documento FPC) de las facturas electronicas creadas en el corte.

```
--Se obtiene el id de la empresa
SELECT @IdEmpresaVarChar = valor FROM tbl_parametros WHERE key_ = 'IdEmpresaFacturacion'
SET @IdEmpresa = CAST(@IdEmpresaVarChar AS INT)

--Se obtiene el documento transaccional para facturacion
SELECT top(1) @IdDocumentoClasificacion = fk_id_doc_factura_electronica
FROM tbl_documentos_clasificacion WHERE fk_id_empresa = @IdEmpresa

--Se obtiene el consecutivo correspondiente
SELECT @ConsecutivoActual = actual FROM tbl_consecutivos
where fk_id_empresa = @IdEmpresa
AND fk_id_documentos_transaccionales = @IdDocumentoClasificacion
AND GETDATE() BETWEEN vigencia_fecha_inicio AND vigencia_fecha_fin

--Obtengo los valores de las facturas
SELECT id_facturacion_electronica AS factura
      ,fk_id_cupo AS cupo
      ,fk_id_cliente AS cliente
      ,valor_total AS valorTotal
      ,(valor_interes + valor_anticipado_interes) AS interes
      ,(valor_iva_interes + valor_anticipado_iva_interes) AS ivaInteres
      ,valor_aval AS aval
      ,valor_iva_aval AS ivaAval
      ,(valor_mora + valor_anticipado_mora) AS mora
      ,(valor_iva_mora + valor_anticipado_iva_mora) AS ivaMora
    INTO #tempFacturasElectronicas
FROM tbl_facturacion_electronica
WHERE fecha_corte = @FechaRealFactura
AND fk_id_comprobante_factura IS NULL
ORDER BY fk_id_cupo ASC
```

Para crear los comprobantes FPC correspondientes a las facturas electronicas del corte se busca en los parámetros el documento, la empresa que factura y el consecutivo actual desde donde se debe continuar la creación de los documentos.

Después se obtiene los valores registrados en la tabla *tbl_facturacion_electronica* al corte en que se está haciendo el proceso.

```
--Aqui realizo los inserts en comprobantes y detalle de comprobantes
SET @ConsecutivoActual = @ConsecutivoActual + 1;

INSERT INTO tbl_comprobantes (fk_id_documento_transaccional, consecutivo, fecha,
fk_id_cliente, fk_id_cupo, valor_total, fk_id_empresa)
VALUES (@IdDocumentoClasificacion, @ConsecutivoActual, @FechaRealFactura,
@idCliente, @idCupo, @valorTotal, @IdEmpresa);

SELECT @IdUltimoComprobante = SCOPE_IDENTITY();

INSERT INTO tbl_comprobante_detalles (fk_id_comprobante, valor_debito, valor_credito,
valor_capital, valor_interes, valor_iva_interes, valor_aval, valor_iva_aval, valor_mora,
valor_iva_mora)
VALUES (@IdUltimoComprobante, @valorTotal, 0, 0, @valorInteres, @valorIvaInteres,
@valorAval, @valorIvaAval, @valorMora, @valorIvaMora)

--Actualizo el campo de comprobantes en la tabla de facturacion electrónica
UPDATE tbl_facturacion_electronica
SET fk_id_comprobante_factura = @IdUltimoComprobante, fk_id_doc_factura_electronica = @ConsecutivoActual
WHERE id_facturacion_electronica = @IdFactura

FETCH NEXT FROM CursorCrearFacturacionElectronica INTO @IdFactura, @IdCupo, @IdCliente, @valorTotal,
@valorInteres, @valorIvaInteres, @valorAval, @valorIvaAval, @valorMora, @valorIvaMora
```

Después se entra a un cursor para crear el comprobante (FPC) y el detalle respectivo por cada factura electrónica registrada, aumentando en cada comprobante el número de consecutivo con el que se está creando el documento.

```
UPDATE tbl_consecutivos
SET actual = @ConsecutivoActual
WHERE fk_id_empresa =  @IdEmpresa
AND fk_id_documentos_transaccionales = @IdDocumentoClasificacion
AND GETDATE() BETWEEN vigencia_fecha_inicio AND vigencia_fecha_fin
```

Cuando finalice el proceso se deberá actualizar en la tabla *tbl_consecutivos* el valor actual con el que quedo creado el último documento (FPC) creado en el proceso.

- Ejecutar el script sp_crear_comprobantes_notas_electronicas.sql que contiene la creación del procedimiento almacenado que realiza los comprobantes (documento NFR) de las facturas electronicas creadas en el corte.

```
--Se obtiene el id de la empresa
SELECT @IdEmpresaVarChar = valor FROM tbl_parametros WHERE key_ = 'IdEmpresaFacturacion'
SET @IdEmpresa = CAST(@IdEmpresaVarChar AS INT)

--Se obtiene el documento transaccional para facturacion
SELECT @IdDocumentoClasificacion = valor FROM tbl_parametros WHERE grupo = 'FacturacionElectronica' AND key_ = 'NotaReclasificacionSaldo'

--Se obtiene el consecutivo correspondiente
SELECT @ConsecutivoActual = actual FROM tbl_consecutivos
where fk_id_empresa = @IdEmpresa
AND fk_id_documentos_transaccionales = @IdDocumentoClasificacion
AND GETDATE() BETWEEN vigencia_fecha_inicio AND vigencia_fecha_fin

--Obtengo los valores de las facturas
SELECT id_facturacion_electronica AS factura
      ,fk_id_cupo AS cupo
      ,fk_id_cliente AS cliente
      ,valor_total AS valorTotal
      ,(valor_interes + valor_anticipado_interes) AS interes
      ,(valor_iva_interes + valor_anticipado_iva_interes) AS ivaInteres
      ,valor_aval AS aval
      ,valor_iva_aval AS ivaAval
      ,(valor_mora + valor_anticipado_mora) AS mora
      ,(valor_iva_mora + valor_anticipado_iva_mora) AS ivaMora
    INTO #tempFacturasElectronicas
FROM tbl_facturacion_electronica
WHERE fecha_corte = @FechaRealFactura
AND fk_id_comprobante_nota IS NULL
ORDER BY fk_id_cupo ASC
```

Para crear los comprobantes NCR correspondientes a las facturas electronicas del corte se busca en los parámetros el documento, la empresa que factura y el consecutivo actual desde donde se debe continuar la creación de los documentos.

Después se obtiene los valores registrados en la tabla *tbl_facturacion_electronica* al corte en que se está haciendo el proceso.

```
--Aqui realizo los inserts en comprobantes y detalle de comprobantes
SET @ConsecutivoActual = @ConsecutivoActual + 1;

INSERT INTO tbl_comprobantes (fk_id_documento_transaccional, consecutivo, fecha,
fk_id_cliente, fk_id_cupo, valor_total, fk_id_empresa)
VALUES (@IdDocumentoClasificacion, @ConsecutivoActual, @FechaRealFactura,
@idCliente, @idCupo, @valorTotal, @IdEmpresa);

SELECT @IdUltimoComprobante = SCOPE_IDENTITY();

INSERT INTO tbl_comprobante_detalles (fk_id_comprobante, valor_debito, valor_credito,
valor_capital, valor_interes, valor_iva_interes, valor_aval, valor_iva_aval, valor_mora,
valor_iva_mora)
VALUES (@IdUltimoComprobante, @valorTotal, 0, 0, @valorInteres, @valorIvaInteres,
@valorAval, @valorIvaAval, @valorMora, @valorIvaMora)

--Actualizo el campo de comprobantes en la tabla de facturacion electrónica
UPDATE tbl_facturacion_electronica
SET fk_id_comprobante_nota = @IdUltimoComprobante, fk_id_doc_nota_electronica = @ConsecutivoActual
WHERE id_facturacion_electronica = @IdFactura

FETCH NEXT FROM CursorCrearFacturacionElectronica INTO @IdFactura, @IdCupo, @IdCliente, @valorTotal,
@valorInteres, @valorIvaInteres, @valorAval, @valorIvaAval, @valorMora, @valorIvaMora
```

Después se entra a un cursor para crear el comprobante (NFR) y el detalle respectivo por cada factura registrada, aumentando en cada comprobante el número de consecutivo con el que se está creando el documento.

```
UPDATE tbl_consecutivos
SET actual = @ConsecutivoActual
WHERE fk_id_empresa =  @IdEmpresa
AND fk_id_documentos_transaccionales = @IdDocumentoClasificacion
AND GETDATE() BETWEEN vigencia_fecha_inicio AND vigencia_fecha_fin
```

Cuando finalice el proceso se deberá actualizar en la tabla *tbl_consecutivos* el valor actual con el que quedo creado el último documento (NFR) creado en el proceso.

- Ejecutar el script (H) que contiene la creación del procedimiento almacenado que realiza los registros de los estados de cuenta de cada cliente.

PENDIENTE DE AJUSTE

1.4 Observaciones y/o Comentarios

En caso tal de que el servicio que realiza el proceso automático falle, se tendría que realizar el proceso rápidamente manual ya que este proceso es importante para seguir con las operaciones respectivas en credi10.

La forma y el orden de ejecutar el proceso manual se puede hacer como en la siguiente imagen, ejecutando de manera manual cada uno de los procedimientos almacenados listados.

```
--Realizar el proceso manual del corte (1 y 16 de cada mes si fallo el servicio)
EXEC sp_saldos_al_corte_op
EXEC sp_debido_cobrar
EXEC sp_crear_valores_facturas
EXEC sp_crear_comprobantes_facturas @FechaCorte = 'fecha corte'
EXEC sp_crear_comprobantes_facturas_electronicas @FechaCorte = 'fecha corte'
EXEC sp_crear_comprobantes_notas_electronicas @FechaCorte = 'fecha corte'
```

2. CASOS DE PRUEBAS UNITARIAS

A continuación, se demuestra las pruebas funcionales hechas en el ambiente de pruebas.

ID	CASO DE PRUEBA	RESULTADO ESPERADO	APROBADO
1	Ejecutar los diferentes procedimientos almacenados de forma automática desde el servicio Windows realizado.	Se realiza el proceso de corte completo.	SI